

LLM Perception of Time

A Seven-Experiment Scoping Study — Methods, Results, and Reproduction

Time Project (sandbox exploration)

Compiled June 11, 2026

Repository: `explorations/time/` Branch: `claude/time-project-experiments-g0k16r`

Abstract

We ask whether large language models perceive time. The question conflates three distinct problems, and we separate them with seven controlled, API-only experiments (no training). **Self-perception (E1, E2, E5, E6, E10):** a model’s apparent ability to estimate its own generation latency is *entirely a proxy for its own output length*. It is well calibrated when latency is output-driven (Spearman $\rho = 0.79$) and collapses to zero when latency is driven by hidden reasoning or input prefill ($\rho \approx 0$); estimating in token space beats seconds ($\rho 0.89$ vs 0.66); and the systematic $\sim 2\times$ length undershoot is mostly fixable in-context (geometric-mean ratio $0.37 \rightarrow 0.76$) for non-reasoning models. **Agency (E3, E4):** agents act on elapsed time only when it is explicit; an agent’s correct time-sensitive decision rate rises from 0.49 (no signal) to 0.80 (text timestamps) to 0.96 (an explicit harness-injected elapsed-time field). The unifying finding: **LLMs have no internal clock — they have a length estimator, and they respond to elapsed time only when it is handed to them.** The recurring boundary across the first six experiments is the *reasoning model*, whose latency hides in tokens it cannot see — but E10 sharpens this: reasoning models retain strong *ordinal* awareness of their own effort (rating ρ up to 0.91) and fail only at *magnitude*, making the boundary a tractable calibration problem rather than a wall. Headline numbers carry 95% bootstrap CIs. This document is self-contained and reproducible: every number below is regenerated by the committed scripts from the committed raw data.

Contents

1	Scope and the three lanes	2
2	Shared infrastructure and reproducibility	2
2.1	Models	2
2.2	The call interface	3
2.3	Environment	3
2.4	Data format and the resume/dedup scheme	3
2.5	How to reproduce	4
2.6	Determinism caveats	4
3	E1 — Self-duration calibration	4
4	E2 — Token/step proxy vs second-space	5
5	E3 — Log-gap probe (are agents timestamp-blind?)	6

6	E4 — Harness clock (sensor vs text timestamps)	7
7	E5 — Latency-decoupled probe (the linchpin validity check)	8
8	E6 — Length-estimation bias correction	9
9	E10 — Reasoning-model hidden-token probe	10
10	Statistical robustness	11
11	Synthesis	12
12	Recommendation and next experiments	13
13	Limitations and process notes	14
A	File manifest	14
B	Experiment summary table	15

1 Scope and the three lanes

“Does an LLM perceive time?” is really three questions that are mechanistically unrelated. A model can be strong at one and hopeless at another:

1. **Temporal reasoning in text** — ordering events, date arithmetic, durations stated in language. Well studied and largely saturated (TimeBench, TRAM, Test-of-Time). We do not pursue it.
2. **Temporal self-perception** — can a model estimate the time/length of its *own* computation? Experiments E1, E2, E5, E6, E10.
3. **Time-aware agency** — does an agent change behaviour based on elapsed time in a transcript? Experiments E3, E4.

The motivation, literature notes and reading list are in `research-notes.md`. The experiment plan is in `experiments.md`; the cross-experiment synthesis in `SUMMARY.md`; and an alternative “train-it-in” research direction in `direction-train-it-in.md`. This report consolidates all of it.

2 Shared infrastructure and reproducibility

2.1 Models

All experiments draw from one fixed roster, defined in `common.py`. Exact API model IDs as used (the study was run on 2026-06-11):

Roster key	Provider	API model ID	Reasoning model?
haiku	Anthropic	claude-haiku-4-5-20251001	no
sonnet	Anthropic	claude-sonnet-4-6	no
opus	Anthropic	claude-opus-4-8	no
gpt4o-mini	OpenAI	gpt-4o-mini	no
gpt4o	OpenAI	gpt-4o	no
gpt5	OpenAI	gpt-5	yes
gpt5.2	OpenAI	gpt-5.2	yes
o4-mini	OpenAI	o4-mini	yes

Reasoning models spend hidden reasoning tokens, so their wall-clock latency decouples from visible output. `common.py` handles the two provider quirks discovered empirically: OpenAI reasoning models require `max_completion_tokens` (not `max_tokens`) and reject a `temperature` argument; `claude-opus-4-8` also rejects `temperature` in this harness (so E4 passes `temperature=None` for opus). Reasoning models have their token budget bumped to ≥ 4096 because hidden reasoning consumes the output budget.

2.2 The call interface

Every experiment imports one helper so provider logic is defined once:

```
from common import call # roster key -> unified call
r = call("sonnet", prompt, max_tokens=1024)
# r: Result(model, text, latency_s, input_tokens, output_tokens, ok, error)
```

`latency_s` is wall-clock measured with `time.perf_counter()` around the network call; it includes network and queueing time, not only generation. Retries use exponential backoff (4 attempts).

2.3 Environment

- Python 3.11; virtualenv at `explorations/time/.venv`. Pinned dependencies in `requirements.txt` (anthropic 0.109.1, openai 2.41.1, numpy 2.4.6, pandas 3.0.3, scipy 1.17.1, matplotlib 3.10.9).
- Secrets via environment variables `ANTHROPIC_API_KEY` and `OPENAI_API_KEY`.

2.4 Data format and the resume/dedup scheme

Each experiment directory `eN-*/` contains: `run.py` (collect raw data), `analyze.py` (compute statistics and figures), `results.jsonl` (one JSON object per API call), `REPORT.md` (self-contained findings), and PNG figures. Collection is *append-only*; `run.py` is idempotent via a `load_done()` check on a logical key (typically `(model, task/scenario, trial, condition)`), so an interrupted run resumes without recomputation.

Known hazard (documented, not silent). This `append + load_done()` scheme is **not safe against concurrent writers**: two `run.py` processes for the same experiment will both pass `load_done()` and write duplicate rows. This actually occurred during the run (see §13). The fix is `dedup.py`, which canonicalizes each `results.jsonl` by its logical key (keeping the last occurrence) before analysis. `analyze.py` scripts also tolerate duplicates, but `dedup.py` guarantees exact counts.

2.5 How to reproduce

From `explorations/time/`:

```
python -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
export ANTHROPIC_API_KEY=... OPENAI_API_KEY=...

# Re-derive every table and figure from the committed raw data (no API calls):
for d in e1-self-duration e2-token-time e3-log-gap-probe \
    e4-harness-clock e5-latency-decoupled e6-length-bias \
    e10-reasoning-tokens; do
    ( cd $d && python analyze.py )
done

# To re-collect raw data (spends API budget). run.py RESUMES from results.jsonl,
# so for a true from-scratch run, move the existing file aside first:
# cd eN-*
# mv results.jsonl results.jsonl.bak # keep a backup; run.py starts fresh
# python run.py # collect; resumable if interrupted
# python ../dedup.py eN-* # only needed if writers ran concurrently
# Quick smoke test of one model/task without a full sweep: python run.py dryrun
# Collection knobs (model ROSTER, N_TRIALS) are at the top of each eN-*/run.py.
```

Re-running `analyze.py` on the committed `results.jsonl` reproduces every statistic in this report exactly. Re-collecting with `run.py` reproduces them up to sampling noise.

2.6 Determinism caveats

Calls use provider-default temperature (not pinned), so re-collection varies trial-to-trial. Absolute wall-clock numbers are harness- and hardware-specific. We therefore lead with *rank* statistics (Spearman ρ , ordering accuracy) and *between-condition* contrasts, which are robust, and treat absolute latencies as distributions (multiple trials, reported coefficient of variation) rather than points.

3 E1 — Self-duration calibration

Question. Can a model estimate its own wall-clock generation time? **Hypothesis (from the literature).** No: estimates overshoot by $4\text{--}7\times$ and ordering is at chance.

Setup. 12 tasks spanning short→long outputs. For each (model $\times$ task $\times$ 3 trials): a PRE estimate (“how many seconds will your response take?”), the real generation (record `latency_s`, `output_tokens`), and a POST estimate (“how long did that take?”). 6 models, 648 calls, 216 units.

Results (Table 1, Fig. 1). The hypothesis is *rejected* for this task set. Estimate/actual geometric-mean ratios sit near 1 (overall 0.81), not $4\text{--}7\times$; ordering accuracy is 0.79–0.95 (chance = 0.5); Spearman ρ_{PRE} is 0.74–0.94, all $p < 0.001$. Trial-to-trial latency noise is small (median CV 0.04–0.14), so the calibration signal is real.

Why the contradiction. The task set varies tasks primarily by *output length*, and for non-reasoning models latency is output-token-dominated. Models can anticipate roughly how long their own output will be, so they predict latency well via an implicit length proxy — not via a

Table 1: E1: per-model calibration. Ratio is geometric-mean estimate/actual.

model	gm ratio PRE	ρ_{PRE}	ordering acc.	ρ_{POST}
opus	0.93	0.94	0.94	0.95
sonnet	0.76	0.85	0.95	0.85
gpt4o	1.04	0.83	0.86	0.77
gpt4o-mini	1.36	0.74	0.86	0.89
haiku	0.86	0.89	0.88	0.76
gpt5.2	0.32	0.74	0.79	0.84
overall	0.81	0.75	—	—

clock. Two tells: gpt5.2 (reasoning) is the worst calibrated ($0.32\times$, it undershoots because hidden reasoning latency is invisible), and undershoots cluster on free-form generations (`code`, `story`). E5 (§7) confirms this mechanism and reconciles the result with the literature.

Verdict. Rejected *for this regime*: models predict their own latency well when it is output-driven (ρ up to 0.94), via a length proxy, not a clock.

4 E2 — Token/step proxy vs second-space

Hypothesis. Output length is something the model has more access to than time, so estimating in tokens should be better-calibrated than in seconds.

Setup. Same task family. Four conditions per (model $\times$ task $\times$ 3): (a) predict seconds; (b) predict output tokens; (c) predict tokens then convert via a measured per-model tokens/sec constant; (d) reason about size first, then predict seconds. 6 models, 864 calls.

Results (Table 2, Fig. 2). Token-space beats second-space: pooled ρ 0.889 vs 0.657 ($\Delta = +0.231$). Per-model, seconds estimates correlate well *within* a model but the scale is wildly model-specific (gpt4o-mini overshoots $6.6\times$, o4-mini undershoots $0.5\times$), which destroys the pooled correlation; tokens normalize this. Models under-predict their own length $\sim 2\times$ (token ratio 0.49). Converting tokens to seconds (c) compounds the undershoot (0.24). Reasoning first (d) does *not* help (0.545).

Table 2: E2: pooled calibration by estimation condition.

condition	ρ (pred vs actual)	gm(pred/actual)
(a) seconds	0.657	2.16
(b) tokens	0.889	0.49
(c) tokens $\rightarrow$ sec	0.829	0.24
(d) reason-then-sec	0.545	1.08

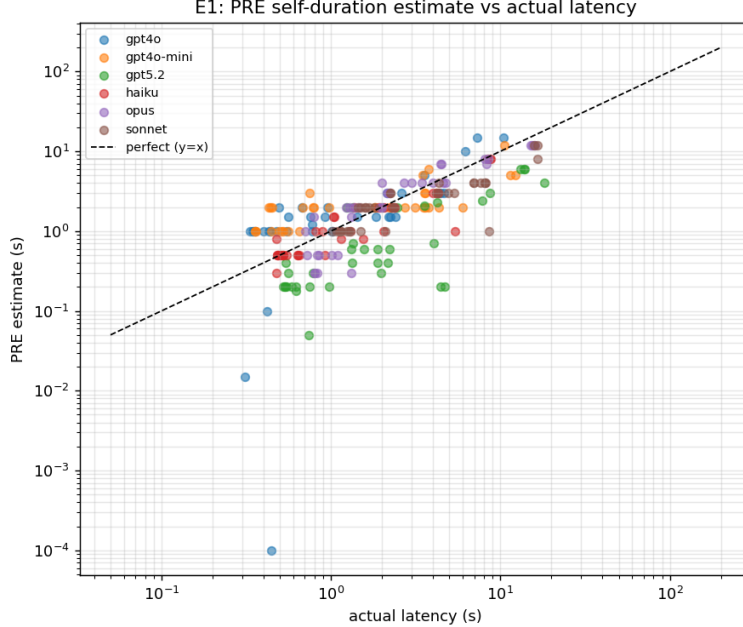


Figure 1: E1: estimated vs. actual latency (log-log), pooled. Tight diagonal structure — models rank and roughly size their own latency.

Verdict. Supported. Models track their own output length, not time; the practical target for any self-estimation is token/step space plus a bias correction for the $\sim 2\times$ undershoot. The approach fails for reasoning models (o4-mini token ratio 0.10), the recurring boundary.

5 E3 — Log-gap probe (are agents timestamp-blind?)

Hypothesis. Agents are blind to elapsed time encoded in a transcript: varying only the timestamp gap will not flip a time-sensitive decision.

Setup. 9 scenarios (stock price, weather, inventory, 2FA code, session token, “wait a bit”, geocode, medication dose, food safety), each a short transcript where an action happened at t_0 and a decision is needed at $t_0 + \text{gap}$. The transcript is held *fixed*; only the gap varies over $\{1\text{s}, 1\text{min}, 1\text{h}, 1\text{day}, 1\text{week}\}$, plus a no-timestamp control. Each scenario declares a human-sensible staleness threshold. One-word forced decision. 5 models, 810 calls. We separate *sensitivity* (does the decision respond to the gap at all?) from *correctness* (does it flip at the right threshold?).

Results (Table 3, Fig. 3). Blindness is *graded*, *not binary*. Sensitivity scales with model strength (opus 1.00 $\rightarrow$ haiku 0.56); when models do attend they are well calibrated (correctness 0.80–0.93). Stripping timestamps (control) changes the decision for no model except opus (33%). The deficit is *attention to the clock*, not temporal judgement.

Verdict. Partially supported, and it refines TicToc’s near-random framing: with a clean single-variable manipulation, strong models *do* respond to gaps and are calibrated; the weak link is inconsistent attention to the clock.

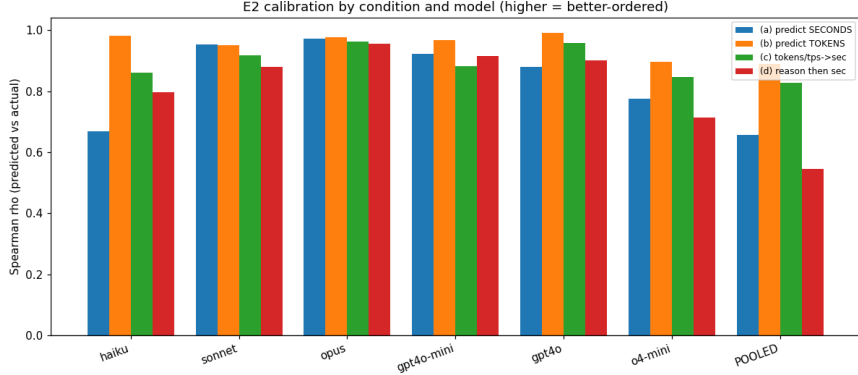


Figure 2: E2: rank correlation of estimate vs. actual, by condition and model. Token space (b) is uniformly high; second space (a) and reasoning (d) are noisier.

Table 3: E3: sensitivity and correctness by model.

model	sensitivity	correctness
opus	1.00	0.86
sonnet	0.78	0.93
gpt5.2	0.78	0.89
gpt4o	0.67	0.80
haiku	0.56	0.82

6 E4 — Harness clock (sensor vs text timestamps)

Hypothesis. Much “temporal blindness” is a missing sensor: an explicit harness-injected elapsed-time field beats burying timestamps in text.

Setup. E3-family time-sensitive decisions under three presentation conditions of the *same* scenario+gap: (a) no time signal; (b) absolute timestamps in the transcript text (model must compute the gap); (c) an explicit harness field, e.g. `[elapsed since last action: 3 hours 12 minutes]`. Gaps straddle each threshold so a correct decision is well defined. 5 models, 1620 calls, 0 failures.

Results (Table 4, Fig. 4). The harness clock wins decisively. Correct-decision rate: 0.494 (none) $\rightarrow$ 0.796 (text) $\rightarrow$ 0.956 (harness). Text over nothing = +0.302; harness over text = +0.159; harness over nothing = +0.461. The lift is largest for weak models and smallest for the reasoning model (o4-mini already computes gaps from text). The cleanest effect is on the fresh side of the threshold, where no-signal models default to “stale” (fresh accuracy 0.23) and the harness clock nearly fixes it (0.94).

Verdict. Supported. A large share of agent temporal blindness is a missing-sensor / tooling problem, addressable at the harness layer without post-training. *Caveat:* the prompt states the staleness rule, so this isolates the *sensor*, not threshold knowledge; the between-condition contrast is the result, not the absolute accuracies (which are optimistic).

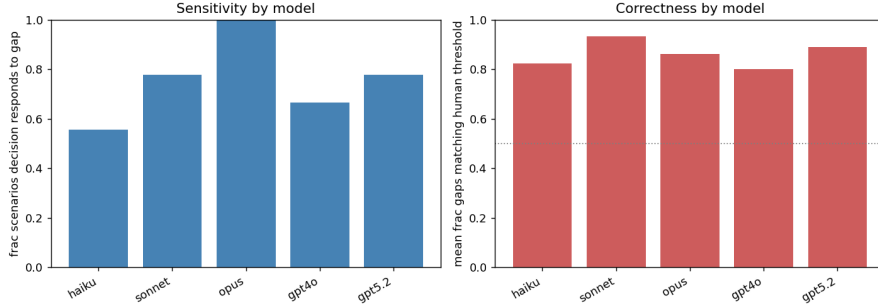


Figure 3: E3: per-scenario sensitivity by model (does the decision change across the gap ladder?). opus reads the clock everywhere; weaker models flatten on extreme-threshold scenarios.

Table 4: E4: correct-decision rate by presentation condition (n=540 each).

condition	correct-decision rate
(a) no time signal	0.494
(b) text timestamps	0.796
(c) harness clock	0.956

7 E5 — Latency-decoupled probe (the linchpin validity check)

Hypothesis. E1/E2’s calibration is a pure output-length proxy: it should survive when latency is output-driven and *collapse* when latency is decoupled from output length.

Setup. Three task families, all logging a PRE seconds-estimate and actual latency: **Type A (reasoning-decoupled)** — short fixed output, varying difficulty (2+2 ... derangement of 8); latency varies via hidden reasoning. **Type B (input-decoupled)** — output pinned to one word, input filler varied 100→30 000 tokens; latency varies via prefill. **Type C (output-driven control)** — output length varies. 5 models, 450 calls.

Results (Table 5, Fig. 5). The prediction holds decisively. Control (C) is strongly calibrated (ρ 0.788, $p < 0.001$), but the reasoning-decoupled (A) and input-decoupled (B) families collapse to no significant correlation *despite* 16× and 9× actual-latency spreads. Even reasoning models fail Type A (gpt5.2 ρ 0.30 n.s., o4-mini −0.36 n.s.). The mechanism is legible in Type B: models emit a near-*constant* time estimate as input grows — e.g. haiku [1.0, 1.0, 1.3, 1.0] seconds for inputs of 100/1k/8k/30k tokens while actual latency rises [0.52, 0.59, 0.71, 0.97]; o4-mini emits [1.0, 1.0, 1.0, 1.0]. A 300× change in input length moves the estimate not at all.

Table 5: E5: pooled ρ (estimate, actual latency) by task type.

type	pooled ρ	gm(est/act)	actual-latency spread
C — output-driven (control)	0.788	1.30	61.9×
A — reasoning-decoupled	−0.067 (n.s.)	2.01	16.2×
B — input-decoupled	−0.231 (n.s.)	1.44	8.9×

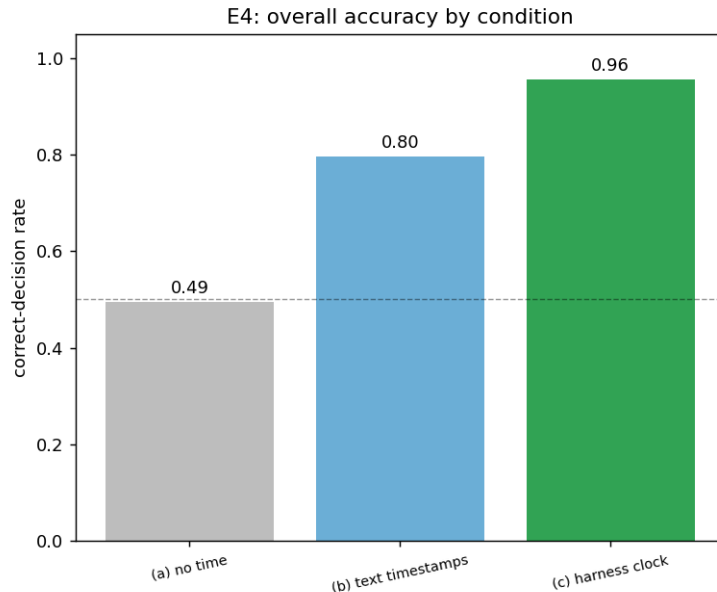


Figure 4: E4: overall correct-decision rate by condition. The sensor (c) closes most of the gap left by text timestamps (b).

Reconciliation. This resolves E1’s apparent contradiction of the literature. There is no contradiction: in length-dominated tasks models look calibrated; in latency-decoupled tasks (closer to what negative-result papers used) they are blind, exactly as reported. The regime is the hidden variable, and E5 makes it explicit.

Verdict. Confirmed. Self-latency calibration is a pure output-length proxy; any “train-it-in” effort must target the latency the model currently cannot see (reasoning tokens, prefill), not output length.

8 E6 — Length-estimation bias correction

Hypothesis. The $\sim 2\times$ output-length undershoot (E2) closes with in-context interventions, no training.

Setup. 10 length-varying tasks. Per (model $\times$ task $\times$ 3): one generation (actual tokens) and three estimate conditions — **bare** (E2 baseline), **anchors** (few-shot token-count calibration table), **self_revise** (predict, then “you tend to under-count, adjust”, then **FINAL**: $\langle n \rangle$). 6 models, 720 calls.

Results (Table 6, Fig. 6). In-context calibration substantially closes the bias while keeping ranking intact. Pooled $\text{gm}(\text{pred}/\text{actual})$ moves 0.37 (bare, $\approx 2.7\times$ undershoot) $\rightarrow$ 0.55 (anchors) $\rightarrow$ 0.76 (self-revise), with ρ holding near 0.83. Per-model nuance is the real story: strong models land near 1.0 under self-revision (opus 0.82, sonnet 0.78), weak models *overcorrect* (haiku $\rightarrow$ 2.01, overshoot), and o4-mini stays stuck (0.08 $\rightarrow$ 0.19).

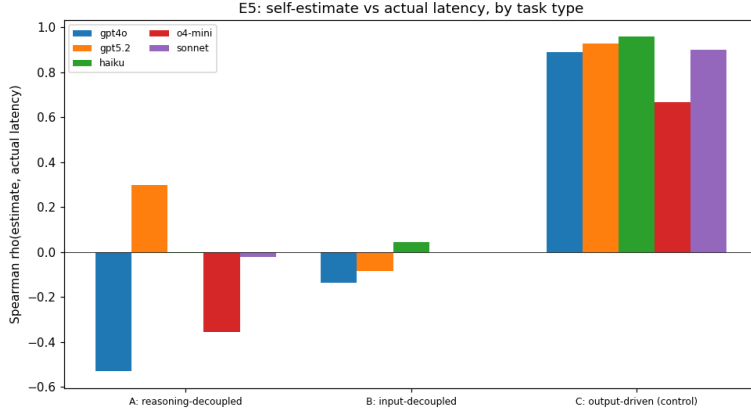


Figure 5: E5: self-estimate vs. actual latency by task type and model. Calibration exists only in the output-driven control (C); it vanishes once latency is decoupled (A, B).

Table 6: E6: pooled length-estimate calibration by condition (1.0 = perfect; <1 = undershoot).

condition	pooled gm(pred/act)	Δ vs bare	pooled ρ
bare	0.37	—	0.842
anchors	0.55	+0.18	0.861
self_revise	0.76	+0.39	0.825

Verdict. Largely supported. Target 1 (length self-estimation) is nearly free for non-reasoning models via a fitted in-context correction; fine-tuning effort should be reserved for the reasoning-model hidden-token problem, the one place in-context fixes fail.

9 E10 — Reasoning-model hidden-token probe

Hypothesis. A reasoning model is blind to its own reasoning cost: it cannot predict how much internal reasoning a problem needs ($\rho \approx 0$). This is the boundary E1, E2, E5, E6 all isolated.

Setup. 14 short-answer problems across a reasoning-difficulty gradient (all with a one-token-ish final answer, so the variable is internal reasoning, not output). Per (model $\times$ task $\times$ 3): **pre_tokens** (estimate, without solving, how many internal reasoning tokens it will need), **pre_effort** (rate 1–10 how much thinking it needs), and the solve, recording *actual reasoning tokens* from usage. Models: o4-mini, gpt5 — the OpenAI reasoning models whose *reasoning_tokens* vary with difficulty and are exposed (gpt5.2 does not reason by default, excluded). 252 calls; actual reasoning spans 0–4757 tokens.

Results (Table 7, Fig. 7). **The hypothesis is wrong, and the truth is more interesting.** Reasoning models have strong *ordinal* awareness of their own reasoning cost but no *magnitude* calibration. The 1–10 effort rating tracks actual reasoning tokens well (pooled ρ 0.77, ordering 0.90; gpt5 ρ 0.91). But token-count estimates are off ~ 5 – $6\times$ (gm 0.17–0.24), gpt5 frequently *refuses* to give a number (parseable for only 8/14 tasks, and wildly inconsistent when it does), and both models badly under-rate the extreme: the hardest task ($\sim 4000+$ reasoning tokens, 5 – $10\times$ the next) drew an effort rating of only 3/10.

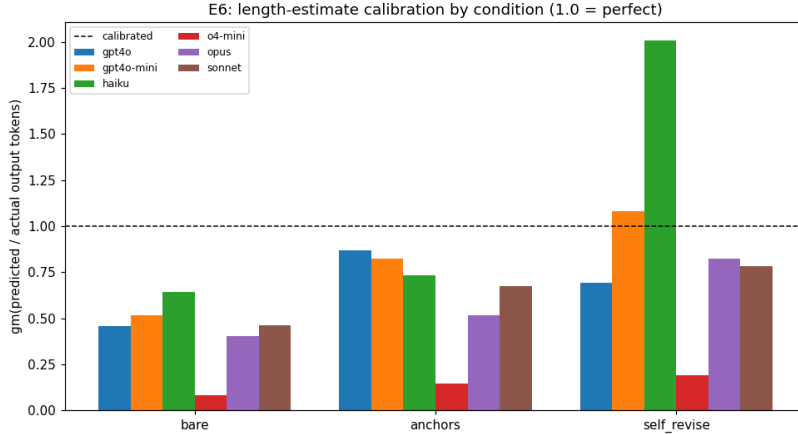


Figure 6: E6: calibration (predicted/actual output tokens) by condition and model; dashed line is perfect calibration. Self-revision pulls strong models to ≈ 1.0 but overshoots haiku and fails to move o4-mini.

Table 7: E10: predicting one’s own reasoning cost (ρ vs actual `reasoning_tokens`).

model	predictor	ρ vs actual	ordering acc.	n
gpt5	effort 1–10	0.908	0.943	14
o4-mini	token estimate	0.796	0.837	14
o4-mini	effort 1–10	0.602	0.867	14
gpt5	token estimate	0.036	0.500	8*
pooled	effort 1–10	0.772	0.901	28
pooled	token estimate	0.403	0.668	22

*gpt5 gave a parseable token number for only 8/14 tasks; its token-estimate ρ is on a thin self-selected subset and is not meaningful. Its effort rating is excellent.

Why this refines the central boundary. E1–E6 framed reasoning models as “blind to hidden reasoning.” E10 sharpens that to *magnitude*-blind, *ordinally*-aware. Asked in the right currency — *how hard is this for you?* not *how many tokens?* — the model answers well. This is structurally the *same* problem E6 fixed for output length: intact ranking, broken scale. So the reasoning-model boundary is a tractable calibration target (a fitted effort→tokens map), not a fundamental wall — the most optimistic update for the “train-it-in” direction.

Verdict. Rejected, with a sharper replacement: reasoning models are ordinally aware of their own reasoning effort (ρ up to 0.91) but cannot calibrate its magnitude. *Caveats:* 14 tasks, 2 models; gpt5 token refusals; reasoning effort is partly stochastic (a distribution, not a point); Anthropic extended-thinking models untested.

10 Statistical robustness

95% confidence intervals on the load-bearing headlines, via cluster bootstrap (`bootstrap-ci.py`) resampling at the correct independent unit — scenarios for E4, tasks for E5, (model,task) units for

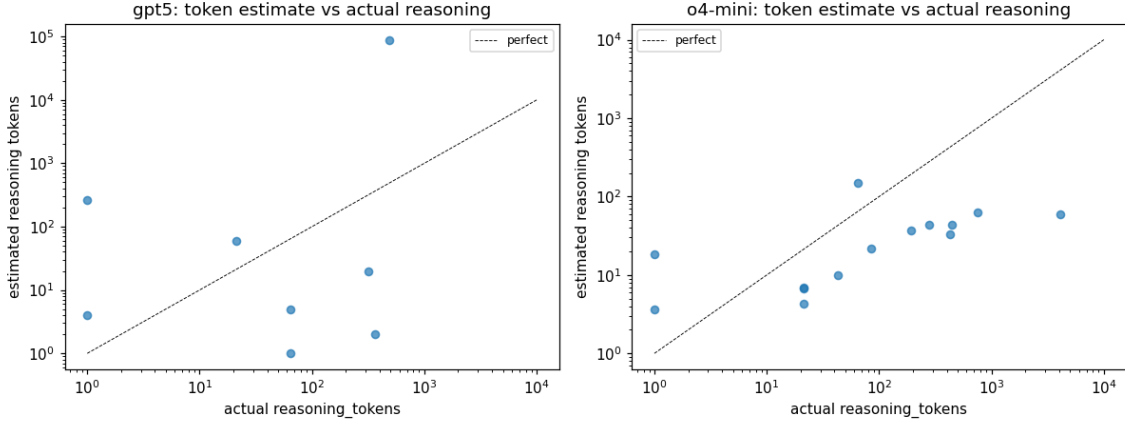


Figure 7: E10: predicted vs. actual reasoning tokens (log-log). Estimates cluster low and flat relative to the wide actual range — magnitude is uncalibrated even where ordering is good.

E1/E2/E6 — rather than individual rows, which would understate uncertainty by ignoring the 3 correlated trials per unit. No new API calls.

Statistic	Point	95% CI
E4 correct rate — no signal	0.494	[0.485, 0.500]
E4 correct rate — text	0.796	[0.741, 0.854]
E4 correct rate — harness clock	0.956	[0.917, 0.983]
E5 ρ — output control (C)	+0.788	[+0.243, +0.835]
E5 ρ — reasoning-decoupled (A)	−0.067	[−0.265, +0.199]
E5 ρ — input-decoupled (B)	−0.231	[−0.495, −0.122]
E1 ρ (self-latency, overall)	0.747	[0.625, 0.835]
E2 ρ — seconds	0.657	[0.499, 0.778]
E2 ρ — tokens	0.889	[0.806, 0.938]
E6 gm — bare	0.37	[0.29, 0.46]
E6 gm — self_revise	0.76	[0.59, 0.95]

The E4 ladder is cleanly separated. E5 confirms the proxy story: the control is robustly positive while both decoupled families exclude positive calibration. E2’s token-vs-second gap is real but narrower than the points suggest (intervals nearly touch). **The wider intervals (E5-C, E2) are limited by task/scenario count, not trial count** — so the principled way to tighten them is more scenarios (E7), not more trials, which is why we did not spend budget raising N from 3.

11 Synthesis

The seven experiments tell one coherent story:

LLMs do not have an internal clock. They have an output-length estimator, and they respond to elapsed time only when it is handed to them explicitly.

Self-perception (E1, E2, E5, E6, E10). Apparent temporal self-awareness is output-length anticipation. It proxies latency whenever latency is length-dominated (E1 ρ up to 0.94; E2 token

space ρ 0.89), and collapses the instant latency hides in unseen reasoning or prefill (E5 $\rho \approx 0$, constant estimates). The length undershoot is mostly an in-context calibration error for non-reasoning models (E6 0.37 $\rightarrow$ 0.76).

Agency (E3, E4). The same absence of a clock means agents act on elapsed time only when it is explicit and loud: text timestamps get partway (E3/E4 0.80) because strong models can do the arithmetic; an explicit sensor gets almost all the way (E4 0.96). The 0.49 $\rightarrow$ 0.96 gap is largely a missing-sensor problem, not a missing-capability problem.

The recurring boundary — and its refinement (E10). Across E1–E6 the reasoning model was the exception: its latency hides in tokens it cannot see, so the length proxy (E1, E2, E5) and every in-context fix (E6) failed on it. E10 probed this directly and *softened* it: reasoning models are not blind to their own reasoning — their 1–10 effort rating tracks actual reasoning-token expenditure well (ρ up to 0.91, pooled 0.77) — they only fail at magnitude (token estimates off $\sim 5\text{--}6\times$; gpt5 refuses them). So the boundary is *ordinal awareness intact, magnitude calibration broken* — the same structure E6 fixed for output length, hence a tractable calibration target rather than a wall. This is the most actionable update from the follow-up round and de-risks the hardest case for the train-it-in direction.

12 Recommendation and next experiments

Two framings are on record and are complementary, not exclusive:

- **“Give it a sensor”** (SUMMARY.md): E4’s harness clock is a cheap, deployable win. Treat it as the *ceiling*.
- **“Train it in”** (direction-train-it-in.md): the absence of a clock is the reason to train the skill in. The research question is how much of the sensor’s gap a model can *internalize* from text alone — internalization fraction = (trained – base)/(ceiling – base).

A note on trainability, corrected from an earlier overstatement: predicting wall-clock seconds is *not* ill-posed within a *fixed* environment. E1’s latency CV of 0.04–0.14 shows the seconds target is reproducible there; the constants (sec/token, overhead) are absorbed by in-environment post-training. Only the *cross-environment* target (from the prompt alone) is ill-posed.

Experiments (detail in experiments.md).

- **E10 — done (§9):** the reasoning-model boundary is ordinal-aware / magnitude-blind, hence a tractable calibration target. This was the cheap, high-value next step and it has been run.
- **E7 — harden + scale E3:** drop the stated staleness rule, add multi-turn, scale to ~ 200 scenarios with held-out families. Prerequisite for E8.
- **E8 — internalization:** SFT (LoRA) a weak and a strong model on text-timestamp transcripts; report the internalization fraction per model (prediction: highest for weak models). The flagship of the train-it-in direction.
- **E9 — sensor-format ablation:** which representation (absolute / elapsed / relative / count-down) carries E4’s lift.

13 Limitations and process notes

Scientific limitations.

- **Small N** (3 trials; 9–14 tasks / scenarios). Trends are clear; tails are thin. Headline numbers now carry 95% bootstrap CIs (§10); the limiting factor on the wider ones is task/scenario count, not trials.
- **Regime-specificity** (E1, E2, E6 use output-length-dominated tasks by design; E5 is the explicit decoupling check).
- **Forced one-word decisions** (E3, E4) and integer-second estimates (E1) are lower bounds on capability.
- **Stated thresholds** (E3, E4) isolate the sensor from threshold knowledge; absolute accuracies are optimistic, between-condition contrasts are the result.
- **Cost-estimate dollars** in each report are token-count $\times$ approximate prices, not billed amounts; total study spend was well within budget ($\sim$ \$10 of a \$50+\$50 cap).

Process notes (so the next run avoids these).

- **Concurrent-writer bug.** Orchestration relaunched `run.py` while an earlier instance was still alive; both passed `load_done()` and produced 492 duplicate rows in E4 (the E5/E6 driver hit the same race transiently but dedups every loop). Root cause: `append + load_done()` is not concurrency-safe. Mitigation: single writer per experiment, and `dedup.py` before analysis. A file lock or a single coordinator is the proper fix.
- **Relative interpreter paths.** `.venv/bin/python` breaks inside a `cd-subshell` (exit 127); use the absolute interpreter path in scripts.
- **Background-job lifetime.** Long collections (slow reasoning models) outlived the job runner and were killed; `_driver_e5e6.py` is a self-healing driver that dedups and relaunches any died collection until expected counts are reached.

A File manifest

Path	Contents
<code>research-notes.md</code>	Literature notes and reading list (seed doc)
<code>experiments.md</code>	Full E1–E10 plan and status
<code>SUMMARY.md</code>	Cross-experiment synthesis and recommendation
<code>direction-train-it-in.md</code>	Counterpoint “train-it-in” research direction
<code>common.py</code>	Shared LLM-call helper (roster, timing, retries)
<code>dedup.py</code>	Canonicalize <code>results.jsonl</code> by logical key
<code>bootstrap.ci.py</code>	95% cluster-bootstrap CIs for headline statistics
<code>_driver_e5e6.py</code>	Self-healing overnight collection driver
<code>requirements.txt</code>	Pinned Python dependencies
<code>eN-*/run.py</code>	Data collection (resumable)
<code>eN-*/analyze.py</code>	Statistics + figures from <code>results.jsonl</code>

Path	Contents
eN-*/results.jsonl	Raw per-call data (canonical, deduped)
eN-*/REPORT.md	Self-contained per-experiment findings
eN-*/*.png	Figures
e4-harness-clock/NOTES.md	Debugging log (opus temperature, dedup)

B Experiment summary table

Exp	Calls	One-line result
E1	648	Self-latency calibration is a length proxy (ρ up to 0.94); not 4–7 $\times$ overshoot.
E2	864	Token space beats seconds (ρ 0.89 vs 0.66); $\sim 2\times$ length undershoot.
E3	810	Timestamp-sensitivity graded by model strength (opus 1.00 $\rightarrow$ haiku 0.56).
E4	1620	Harness clock 0.96 > text 0.80 > none 0.49.
E5	450	Calibration collapses when latency is decoupled ($\rho \approx 0$); confirms length proxy.
E6	720	Self-revision closes the undershoot 0.37 $\rightarrow$ 0.76 (non-reasoning models).
E10	252	Reasoning models are ordinally aware (effort ρ up to 0.91) but magnitude-blind.